



API design principles

September 2026

Recap day 2



Last time: HTTP methods as a contract, idempotency



Status codes, error handling, unit tests



Everyone built full CRUD with error handling



Today: zooming out to the design level

Today's agenda



API design principles: resources, naming, nesting



Versioning and backward compatibility



HATEOAS: theory vs practice



Integration testing: proving the full contract



Section 1

API design principles in depth

An API is a contract



An API is a shared agreement, not just endpoints



Good design reduces confusion



Bad design creates brittle clients



Strong contracts are easier to debug

Resource modeling



- **Think in nouns, not verbs**
- URLs describe resources
- HTTP methods describe actions
- The method already says what happens

Bad vs good

GET /getCustomer?id=42



GET /customers/42

POST /createOrder



POST /orders

GET /listAllActiveUsers



GET /users?status=active

DELETE /deleteUser/42



DELETE /users/42

When actions are a valid exception

- Not every operation maps cleanly to a resource
- Some actions are real business operations

Examples:

- `/payments/123/capture`
- `/orders/456/cancel`



Collections and sub-resources



Collections are common and clear



Sub-resources model real relationships



`/customers/42/orders` is a real relationship



Nesting should mirror the domain



How deep is too deep?

`/customers/42/orders` → Fine

`/customers/42/orders/99/items` → Still fine

`/customers/42/orders/99/items/7/discounts/2` → Too deep

- Deep nesting becomes noisy
- Every extra level adds coupling to the URL
- Prefer flattening once it gets deep

Mini exercise 1

API review

- Review endpoints
- Identify resource vs action
- Spot naming issues
- Discuss trade-offs





Section 2

Naming, versioning & compatibility

Consistent naming conventions



Use consistent terms across URLs and JSON



Pick singular or plural, don't mix



Use clear field names: `id`, `createdAt`, `status`



Avoid unclear abbreviations

Dates, times and the ISO-8601 rule

Good:

```
"2026-08-25T14:30:00Z"
```

Bad:

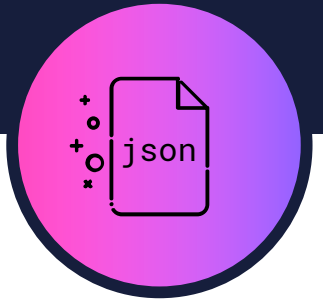
```
"25/08/2026 2:30 PM"
```

Bad:

```
"August 25th, 2026"
```

- Pick one standard: ISO-8601
- Always include a timezone
- Avoid locale-formatted dates entirely

```
{ "middleName": null }  
{ "phoneNumber": "+31 6 1234 5678" }
```



Null vs. missing: a semantic trap

- **null** means "field exists, no value"
- A missing field can mean "not supported" or "not loaded"
- Pick one meaning and be consistent

Versioning: why and when



APIs evolve over time



Not every change is a breaking change



Versioning is a tool for breaking changes



Overusing versions creates maintenance overhead

Dates, times and the ISO-8601 rule

- URL versioning is visible and simple
- Header versioning keeps URLs clean
- Both are common in production APIs

URL versioning:

```
GET /v1/customers/42
```

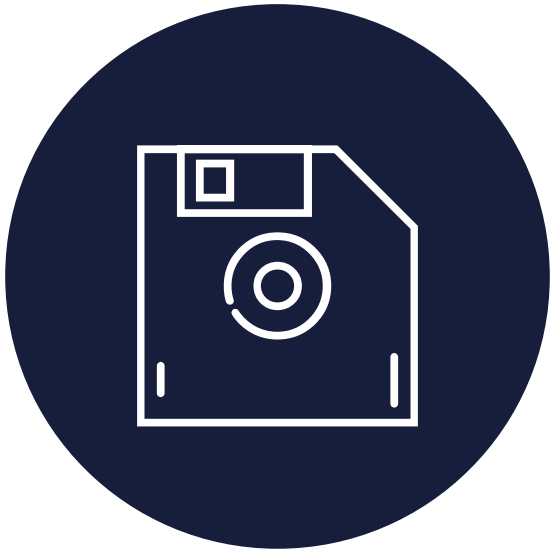
Header versioning:

```
GET /customers/42
```

Accept:

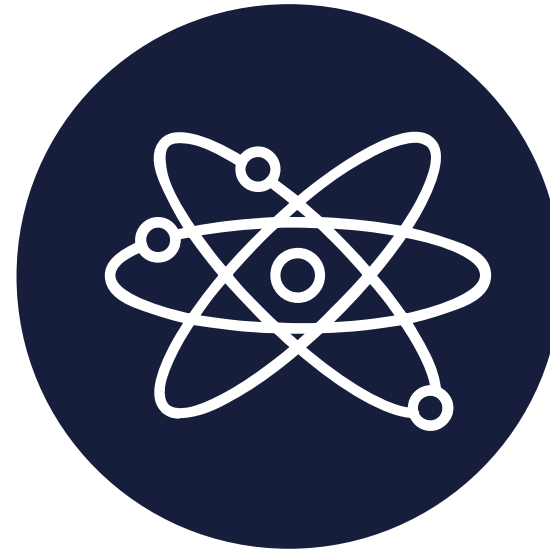
```
application/vnd.acme.v1+json
```

Choosing a versioning strategy



Small, stable APIs

Clear semantic may be enough



Large systems

Plan for controlled evolution

Choosing a versioning strategy



Consider your actual client base



Document the deprecation path

Backward compatibility

Safe vs. risky changes

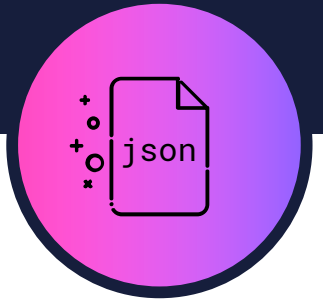
Usually safe

- Adding an optional field
- Adding a new endpoint
- Adding a new enum value (if clients ignore unknowns)

Usually breaking

- Removing a field
- Renaming a field
- Changing a field's type
- Changing status codes for existing behavior

```
{
  "id": 42,
  "name": "Alice",
  "_links": {
    "self": { "href": "/customers/42" },
    "orders": { "href": "/customers/42/orders" }
  }
}
```



HATEOAS: Theory vs. practice

- HATEOAS: hypermedia as the engine of application state
- Links let clients discover related resources
- In practice, documentation often does this job instead

A note on internal conventions



Every team accumulates its own conventions



REST principles are guidelines, **not** laws



Context and team agreements matter too



Consistency within your system is the real goal



Bonus section

Topicus internal framework Cobra

Cobra-rest

Topicus framework, in production since 2013 (the backend behind EduArte & KeyHub)

All three traditions say "the server decides what you may do", the difference:

- Hypermedia REST: models a **state machine**
- Modern pragmatic REST: models a **data schema**
- cobra-rest: models an **access-control matrix**



Ship the authorization result

On every response, for every object, cobra computes what *this user* may do to *this object*, and puts it on the wire

So:

- per object, per user, per request



```
{
  "$type": "persoon.RDeelnemer",
  "links": [
    { "rel": "self",      "href": ".../deelnemer/1234" },
    { "rel": "identiteit", "href": ".../persoon/9876"  }
  ],
  "permissions": [
    { "type": "persoon.RDeelnemer",
      "operations": ["READ", "UPDATE"] }
  ],
  "deelnemernummer": 1234
}
```



The cobra way

- \$type: a type discriminator (JSON:API's idea)
- Links: identity only, no navigation (the whole framework defines just three relation names)
- Permissions: the authorization result, per object
- = HAL's _links + JSON:API's type + a permissions array neither has

Signature mechanisms



Query params are the search filter



Singular resource names: `deelnemer/1234`



Range-based paging: `Range: items=100-199 >> 206`
+ `Content-Range`



Versioning by recorded diff

Example walkthrough



- Let's make it practical

Mini exercise 2

- Review versioning strategies
- Find unclear semantics
- Identify breaking changes
- Choose a safe fix





Section 3

API testing part 3: Integration tests

Why integration tests?



Unit tests prove logic
Integration tests prove contracts



HTTP routing and serialization matter



Real clients see the whole response

What integration tests cover



Routing correctness



Response status codes



JSON serialization



Headers and validation



Error response shape

In-memory databases for testing

- Run the real API against a real database
- Use an in-memory database instead
- Fast, isolated, no external dependencies
- Every test run starts from a clean state



Rest Assured in a nutshell



A library for testing HTTP APIs in Java



Fluent syntax: **given**, **when**, **then**



Assert status codes, bodies, and headers



Reads almost like plain English

Rest Assured example



```
given()
    .contentType(ContentType.JSON)
.when()
    .get("/customers/42")
.then()
    .statusCode(200)
    .body("name", equalTo("Alice"))
    .header("Content-Type", containsString("application/json"));
```

Unit tests vs integration tests

- Unit test: "is my service logic correct?"
- Integration test: "is my API client-ready?"
- Different layers, different questions



Contract tests vs happy-path tests



A happy-path test proves one scenario works



A contract test proves the agreed behavior holds



Contract tests include the unhappy paths too

Testing pyramid in API work

- Many unit tests
- Fewer targeted integration tests
- E2E only for critical flows
- Balance speed and confidence



Mini exercise 3

Which test fits?

- Match scenario to test type
- Unit, integration, or E2E
- Explain the reason



Mini exercise 4

Fix the failing assertion

- Review a test written at the wrong level
- Spot what it actually verifies
- Rewrite the assertion correctly



Demo

First integration test

- Start the API locally
- Hit a real endpoint
- Assert status code
- Assert JSON and headers





Morning recap



Resource modeling: nouns, not verbs



Naming, dates and null semantics



Versioning and backward compatibility



HATEOAS: theory vs. practice



Integration tests prove the real contract

This afternoon



Review API design



Refactor weak contracts



Add integration tests



Bonus: pagination and filtering